

MUNICIPAL WASTE COLLECTION SCHEDULING & ROUTE OPTIMIZATION SYSTEM

A Python-Based Prototype for Civic Waste-Management Operations

Agenda

- 1. Problem Statement
- 2. Objective
- 3. Requirements & Environment
- 4. Design / Proposed Solution
- 5. Algorithm / Pseudocode / Flowchart
- 6. Implementation Overview
- 7. Test Cases & Expected Results
- 8. Sample Execution Output
- 9. Analysis & Discussion
- 10. Conclusion
- 11. Individual Contribution
- 12. References

Why Manual Waste-Collection Management Fails

- Data inconsistency — zone names, truck IDs and timings recorded differently across ward offices
- Inefficient search — locating a truck's zone or a zone's last collection date takes significant time
- Missed pickups — zones skipped with no systematic tracking, causing garbage pile-up and complaints
- Unbalanced truck load — some trucks serve far more stops than others, causing delays and overtime
- No performance analysis — high-missed-pickup zones, truck workload and complaint trends stay invisible
- Manual, redundant, error-prone reporting and scheduling across paper logs and registers

Project Objective

Design, implement and evaluate a Python-based prototype that lets a municipal corporation manage waste-collection zones, truck schedules, and complaint/missed-pickup records efficiently — and generate meaningful civic performance reports.

Register

Zones and trucks with structured, consistent records

Schedule

Assign trucks to zones by day/shift with load limits

Track

Log missed pickups and complaints, then resolve them

Analyze

Report workload, coverage and complaint trends

Requirements & Development Environment

Functional Requirements

- Register zones (ID, area, ward, collection day) and trucks (ID, capacity, driver)
- Assign trucks to zones for a given day/shift
- Log missed pickups and complaints; mark them resolved
- Search zones, trucks and complaints by name, ward or ID
- Generate reports: missed pickups, workload, resolution trend, coverage
- Save/load data via CSV; handle invalid input without crashing

Environment & Tools

- Language: Python 3.x
- IDE: VS Code / PyCharm / IDLE
- Version control: Git & GitHub
- Core libraries: csv, datetime, statistics
- Testing: manual test cases / assertions
- Target scale: 300 zones, 100 truck schedules

Data Structure Design

Entity	Structure	Why
Zones	Dictionary — zone_id : {area, ward, truck_id, last_collected}	O(1) lookup by Zone ID
Truck Schedules	Dictionary — truck_id : {route: [...], shift}	Fast access to a truck's route & shift
Complaints / Missed Pickups	List of dicts — {zone_id, date, status}	Preserves chronological order
Ward & Zone Type	Tuple — (ward_no, zone_type)	Fixed data, protected from accidental change
Shift Types / Wards	Set — {Morning, Afternoon, Evening}	Guarantees uniqueness

System Architecture — Program Flow



Data persists throughout via CSV files (zones.csv, truck_schedule.csv, complaints.csv), loaded at startup and saved on exit or on demand.

Missed-Pickup Detection Algorithm

```
FUNCTION check_missed_pickups(scheduled_day):  
    missed = []  
    FOR zone_id, details IN zones.items():  
        IF details['collection_day'] == scheduled_day:  
            IF details['last_collected'] != today():  
                missed.append(zone_id)  
                complaints.append({  
                    'zone_id': zone_id, 'date': today(),  
                    'status': 'Missed Pickup'  
                })  
    RETURN missed
```

Scans every zone scheduled for the day and auto-logs a complaint if it was not collected.

Truck Assignment & Load-Balancing Algorithm

```
FUNCTION assign_truck_to_zone(truck_id, zone_id, max_zones_per_truck=10):  
    IF truck_id NOT IN truck_schedule:  
        truck_schedule[truck_id] = {'route': [], 'shift': 'Morning'}  
    current_load = LENGTH(truck_schedule[truck_id]['route'])  
    IF current_load >= max_zones_per_truck:  
        RAISE TruckOverloadError  
    truck_schedule[truck_id]['route'].append(zone_id)  
    zones[zone_id]['truck_id'] = truck_id  
    RETURN True
```

Enforces a per-truck capacity cap and raises a custom exception instead of silently overloading a route.

Modular Implementation Plan

zone_manager.py

Zone CRUD operations

truck_manager.py

Truck registration & schedule assignment

complaint_manager.py

Complaint & missed-pickup logging

analytics.py

Performance analysis functions

reports.py

Report generation

file_handler.py

CSV read / write operations

main.py

Main menu & program flow

Minimum Test Scenarios

#	Scenario	Expected Result
TC1	Register a new zone with valid details	Zone added successfully
TC2	Assign a truck within its capacity limit	Zone added to truck's route successfully
TC3	Assign a truck beyond max zone capacity	TruckOverloadError raised & handled gracefully
TC4	Missed-pickup check for an uncollected zone	Zone flagged; complaint auto-logged
TC5	Mark a logged complaint as resolved	Status updated with resolution date
TC6	Search zones by ward number	Matching zone records returned
TC7	Load a missing CSV file	FileNotFoundException handled gracefully

Expected Output Format (Illustrative)

```
===== Municipal Waste Collection System =====  
1. Register Zone    2. Register Truck    3. Assign Schedule  
4. Log Complaint    5. Run Reports        6. Exit  
Select option: 5  
  
-- Missed Pickup Report (2026-08-30) --  
Zone Z07 (Velachery, Ward 14) -> MISSED [Complaint logged]  
Zone Z12 (Adyar, Ward 09)      -> Collected on time  
  
-- Truck Workload Summary --  
T01: 8 zones | T02: 6 zones | T03: 10 zones (near capacity)
```

Actual run screenshots (source code, console output, test execution) should be inserted here from the working implementation.

Analysis, Trade-offs & Justification

Data Structure Choice

- Dictionary for zones -> $O(1)$ lookup vs $O(n)$ list scan
- Tuple for (ward, zone_type) -> protects fixed data from mutation
- Set for shift types -> guarantees uniqueness automatically

Function & Control-Flow Design

- Separate functions per concern -> easier testing & reuse
- if/elif/else drives missed-pickup & overload checks
- for/while loops scan zones and process complaint logs

Exception Handling

- Custom TruckOverloadError improves reliability
- try/except/finally guards file I/O and invalid input
- Prevents crashes from missing files or bad zone IDs

Broader Considerations & Risk Mitigation

Risk	Mitigation
Citizen data privacy (names/contact exposed)	Access controls; restrict details to authorized staff
Service inequity across wards	Standardized scheduling rules; equal frequency for all zones
Missed-pickup detection errors	Validate entries; cross-check schedule vs. actual logs
Accidental data loss	Regular backups; delete confirmation; frequent saves

Conclusion

- Successfully designed a modular system for zone, schedule and complaint management
- Applied CO1 (functions/modules), CO2 (control flow & string handling), and CO3 (dictionaries, lists, tuples, sets)
- Missed-pickup detection, truck load-balancing, and civic reporting work as intended at prototype scale
- Limitations: no GPS tracking, no concurrent access, no GUI/mobile interface yet
- Future improvements: route-optimization algorithms, citizen complaint app, web-based staff dashboard

Individual Contribution of Group Members

Member Name	Module(s) Owned	Key Contribution
[Name 1]	zone_manager.py, truck_manager.py	[Describe specific contribution]
[Name 2]	complaint_manager.py, analytics.py	[Describe specific contribution]
[Name 3]	reports.py, file_handler.py	[Describe specific contribution]
[Name 4]	main.py, testing & documentation	[Describe specific contribution]

Fill in actual names, roles and specific contributions for your group before submission.

References

- Python Documentation (2026). Python 3.x Library Reference. docs.python.org/3/
- Downey, A. B. (2016). Think Python: How to Think Like a Computer Scientist (2nd ed.). O'Reilly Media.
- Severance, C. (2016). Python for Everybody: Exploring Data Using Python.
- Van Rossum, G., & Drake, F. L. (2011). An Introduction to Python.
- General municipal solid-waste management guidelines (collection scheduling & service-level concepts).
- AI-assisted tools used: [e.g., GitHub Copilot / Claude for debugging and documentation].